

LESSON 10

Interview Strategy & Mock Problems

How to think, talk, and solve problems under pressure.

- 10.1 The 4-Step Framework
- 10.2 What to Say When Stuck
- 10.3 Pattern Recognition Cheat Sheet
- 10.4 Time Complexity Quick Reference
- 10.5 Full Mock Problem Walkthrough

10.1 The 4-Step Interview Framework

!! STEP 1: UNDERSTAND (2 min)

- Repeat the problem in your own words
- Ask: "What should I return? Index or value?"
- Ask: "Can there be duplicates? Negative numbers?"
- Ask: "Is the array sorted? What's the size range?"
- Walk through the given example by hand

STEP 2: PLAN (3-5 min)

- Say brute force first: "I could check every pair - $O(n^2)$ "
- Then optimize: "But with a hash map I can do $O(n)$ "
- Explain WHY: "Because lookup in hash map is $O(1)$ "
- Get interviewer's approval before coding

STEP 3: CODE (10-15 min)

- Clean variable names (not i,j,k everywhere)
- Talk while coding: "Now I'm iterating through..."
- Don't delete and rewrite - cross out and continue
- Handle edge cases at the top: if not nums: return 0

STEP 4: TEST (3-5 min)

- Trace through your code with the given example
- Think of edge cases: empty, one element, all same, sorted
- Fix bugs calmly: "I see an off-by-one here, let me fix it"

10.2 What to Say When Stuck

GOOD things to say:

"Let me think about what data structure fits here..."
"This looks like it could be a [two pointer/hash map] problem"
"Can I sort this first to make it simpler?"
"What if I used a hash map to remember what I've seen?"
"Let me start with brute force and optimize from there"
"I think BFS would work here because I need shortest path"
"This reminds me of the sliding window pattern"

NEVER do:

- Stay silent for > 30 seconds
- Jump to coding without a plan
- Say "I don't know" and give up
- Write code without explaining
- Pretend to know something you don't

If truly stuck:

"I'm stuck on X part. Could you give me a hint on the approach?" (Asking for hints is NORMAL and expected)

10.3 Pattern Recognition Cheat Sheet

If Problem Says...	Think...	Pattern
--------------------	----------	---------

Sorted array + find pair	Two pointers	Opposite direction
Remove/filter in-place	Two pointers	Slow/Fast
Contiguous subarray size k	Sliding window	Fixed window
Longest/shortest with condition	Sliding window	Variable window
Count frequency / duplicates	Hash map	Counting
Find pair summing to X	Hash map	Complement lookup
Group similar items	Hash map	Key -> list
Shortest path (equal weight)	BFS	Queue + visited set
Shortest path (diff weights)	Dijkstra / A*	Priority queue
Explore all paths / islands	DFS	Stack / recursion
Search sorted data	Binary search	Half the search space
Overlapping intervals	Sort by start	Merge / sweep line
Top K / Kth element	Sort or Heap	heapq.nlargest
All subsets/permutations	Backtracking	Choose-explore-unchoose

10.4 Time Complexity Reference

O(...)	Name	Example	1K items	1M items
O(1)	Constant	Hash lookup	instant	instant
O(log n)	Logarithmic	Binary search	~10 ops	~20 ops
O(n)	Linear	Single loop	1K ops	1M ops
O(n log n)	Log-linear	Sorting	~10K ops	~20M ops
O(n ²)	Quadratic	Nested loops	1M ops	1T ops (too slow!)
O(2 ⁿ)	Exponential	All subsets	impossible	impossible

10.5 Mock Problem: Complete Walkthrough

"Given a list of integers and a target, return indices of two numbers that add up to target."

STEP 1 - UNDERSTAND:

"So I need to find two different numbers in the list that add up to target, and return their indices, correct?"
"Can there be multiple valid pairs? Should I return first?"
"Can I use the same element twice?" (Usually no)

Example: nums=[2,7,11,15], target=9
By hand: 2+7=9. Indices [0,1]. Got it.

STEP 2 - PLAN:

"Brute force: check every pair with nested loops. $O(n^2)$."
"Better: use a hash map. For each number, I calculate what I need (target-num), and check if I've seen it. One pass through the array. $O(n)$ time, $O(n)$ space."
"Shall I code the optimized version?"

STEP 3 - CODE:

```
def two_sum(nums, target):  
    seen = {} # maps value -> index  
    for i, num in enumerate(nums):  
        complement = target - num  
        if complement in seen:  
            return [seen[complement], i]  
        seen[num] = i  
    return []
```

STEP 4 - TEST:

nums=[2,7,11,15], target=9
i=0, num=2: comp=7, seen={} -> not found -> seen={2:0}
i=1, num=7: comp=2, seen={2:0} -> found! return [0,1]

Edge cases:

- Empty array -> returns []
- No valid pair -> returns []
- Negative numbers -> works fine (complement handles it)

"Time: $O(n)$, Space: $O(n)$. Done!"

**You've got this, Anushka. Own your thesis. Think out loud.
Start brute force, then optimize. The rest is practice.**